

DSA STUDY BLUEPRINT SERIES

Time & Space Complexity Blueprint

Algorithm Analysis and Big-O Notation

Section 05 • C++ Core Data Structures & Algorithms

Understanding Big-O Notation

Measuring how run time and memory scale with input size

What is Big-O?

- Represents the **upper bound** of an algorithm's growth rate.
- Ignores constant factors and lower-order terms.
- Focuses on behavior as input size N approaches infinity.

Why Ignore Constants?

- Hardware differences (CPU, RAM) change execution times.
- Big-O abstracts hardware; a $O(N)$ algorithm will always outperform a $O(N^2)$ algorithm for sufficiently large N .

Common Complexity Classes

Analyzing scaling behaviors from best to worst

Notation	Name	Scaling Behavior	Example Algorithm
$O(1)$	Constant	Time is fixed, independent of input size.	Array index access
$O(\log N)$	Logarithmic	Input size divides in half each step.	Binary Search
$O(N)$	Linear	Time scales directly with input size.	Linear Search, Kadane's
$O(N \log N)$	Linearithmic	Scales slightly worse than linear.	Merge Sort, Quick Sort
$O(N^2)$	Quadratic	Nested iterations over input.	Bubble Sort, Matrix summation

Execution Limits Rule of Thumb: C++ can process roughly **10^8 operations per second**. An $O(N^2)$ algorithm will time limit exceed (TLE) on LeetCode for inputs where $N > 10^4$.

Space Complexity Analysis

Evaluating memory overhead during execution

Auxiliary Space vs. Total Space

- **Total Space:** Memory taken by input + auxiliary memory.
- **Auxiliary Space:** Extra/temporary space allocated by the algorithm itself.
- Algorithms are evaluated based on **Auxiliary Space** complexity.

Common Space Classes

- **$O(1)$:** In-place modifications (e.g. Swapping two elements in an array).
- **$O(N)$:** Allocating extra arrays/vectors equal to input size (e.g. Hash maps).
- **$O(\log N)$ or $O(N)$:** Stack memory overhead in recursion.